# Tablice, lista ocen i statystyki

Rozbudujesz szkolnego asystenta o listę ocen i jej podsumowanie. Tablice poznasz od podstaw. Wykorzystasz znane już funkcje, warunki i pętlę for.

**Efekt lekcji**
Użytkownik dodaje oceny 1-6 i usuwa ostatnią ocenę. Strona pokazuje listę, liczbę ocen, średnią, najwyższą ocenę oraz liczbę ocen co najmniej 4. Każda zmiana listy od razu aktualizuje podsumowanie.

## Twój punkt startowy

Skopiuj katalog **lesson5/start** do własnego folderu roboczego. Otwórz index.html w przeglądarce i script.js w edytorze. Po zapisaniu zmian odświeżaj stronę. Podczas pracy miej otwartą konsolę przeglądarki.

Gotowe są oba kalkulatory, skala ocen, funkcja formatNumber(), przykłady oraz pętla uzupełniająca tabelę procentów. W addGrade() działa pobranie liczby i walidacja braku wyboru. Obsługę tablic i statystyk dopiszesz w miejscach TODO.

Wszystkie elementy strony już istnieją w HTML. Wykonujesz zadania w **script.js**. Wygląd zapewnia Bootstrap pobierany z internetu.

## Rozgrzewka: znany kod

**1. Sprawdź pętlę tabeli.**
Kliknij „Uzupełnij tabelę”. Powinno pojawić się 11 ocen dla procentów 0-100. Dla 40% ocena wynosi 2, a dla 100% wynosi 6.

**2. Sprawdź pusty wybór.**
Kliknij „Dodaj ocenę” bez wybrania oceny. Powinien pojawić się komunikat „Wybierz ocenę od 1 do 6.”.

**3. Uzupełnij małą poprawkę.**
W addGrade(), za blokiem if z return, wyczyść textContent elementu grade-message. Po błędzie wybierz 3 i kliknij dodawanie: komunikat ma zniknąć. Sama lista zacznie działać po dalszych zadaniach.

Dane tej lekcji są przechowywane w pamięci. Odświeżenie strony rozpoczyna pracę z pustą listą.

01 / NOWY TEMAT

# Co to jest tablica?

**Tablica** przechowuje wiele wartości w określonej kolejności pod jedną nazwą. Puste nawiasy kwadratowe tworzą pustą tablicę. W przykładach używamy innych danych niż w projekcie.

```
const temperatures = [];
const scores = [10, 20, 15];
console.log(scores[0]); // 10
console.log(scores[1]); // 20
console.log(scores.length); // 3
```

**Indeks** to numer miejsca w tablicy. Pierwszy indeks wynosi 0, drugi 1. Zapis scores[1] odczytuje drugi element. **length** to liczba elementów, a nie indeks ostatniego. Dla trzech elementów indeksy to 0, 1, 2.

| Zapis | Znaczenie dla scores = [10, 20, 15] |
|---|---|
| scores[0] | Pierwszy element: 10 |
| scores[2] | Ostatni element: 15 |
| scores.length | Liczba elementów: 3 |
| scores[3] | Brak elementu: undefined |

## const i wspólna lista

const nie pozwala przypisać zmiennej innej tablicy. Można jednak zmieniać zawartość tej samej tablicy: dopisywać i usuwać elementy. Dzięki temu lista może rosnąć po kolejnych kliknięciach.

```
const scores = [];
scores.push(10); // dozwolone: zmiana zawartości
// scores = []; // niedozwolone: nowe przypisanie do const
```

## Zadanie 1. Przygotuj magazyn ocen

W miejscu TODO **poza funkcjami** utwórz jedną pustą tablicę o nazwie **grades**, używając const. Będzie dostępna dla funkcji addGrade, removeLastGrade, renderGrades i renderStats.

Nie twórz jej wewnątrz addGrade(): wtedy każde kliknięcie zaczynałoby od nowej pustej listy. W konsoli sprawdź grades.length - na początku ma wynosić 0.

**Sprawdź zrozumienie**
Jak odczytasz drugi element? Jaki będzie ostatni indeks tablicy z pięcioma elementami? Odpowiedzi: [1] oraz 4. Zapis [] tworzy tablicę, natomiast [indeks] po nazwie służy do odczytu elementu.

# Dodaj wartość i pokaż listę

## push(), join() i length

```
const scores = [];
scores.push(10);
scores.push(20);
console.log(scores); // [10, 20]
console.log(scores.join(', ')); // tekst: 10, 20
console.log(scores.length); // 2
```

**push(wartość)** dopisuje element na końcu tablicy. **join(separator)** łączy elementy w jeden tekst, rozdzielając je podanym tekstem. Nie zmienia tablicy. **length** odczytuje jej aktualną długość; zapisujemy je bez nawiasów ().

Nie przypisuj wyniku push() do grades: push() zwraca nową długość, a nie tablicę. Wystarczy osobne wywołanie metody na istniejącej tablicy.

## Elementy gotowego HTML

| id | Zastosowanie |
|---|---|
| grade-value | Select; jego .value zawiera tekst wybranej oceny |
| grade-message | Komunikat błędu lub pusty tekst |
| grade-list | Tekst listy, np. 3, 4, 5 lub Brak ocen |
| grade-count | Liczba elementów tablicy |

## Zadanie 2. Napisz renderGrades()

**1. Rozpoznaj pustą tablicę.**
Jeżeli grades.length === 0, wpisz „Brak ocen” do textContent elementu grade-list.

**2. Obsłuż niepustą listę.**
W else wpisz do grade-list tekst uzyskany z grades.join(), rozdzielając oceny przecinkiem i spacją.

**3. Pokaż licznik.**
Po if/else wpisz grades.length do textContent elementu grade-count. Licznika nie zwiększaj ręcznie.

Ta funkcja **wyświetla stan danych**. Nie dodaje ani nie usuwa ocen. Wywołaj w konsoli renderGrades() - dla nowej tablicy ma pokazać „Brak ocen” i 0. Po wykonaniu zadania 3 sprawdzisz również listę niepustą.

# Dokończ dodawanie oceny

Przycisk „Dodaj ocenę” wywołuje gotową funkcję addGrade(). Pobranie danych i warunek błędu już są w starterze:

```
const value = Number(
  document.getElementById('grade-value').value
);
if (value < 1 || value > 6) {
  // W starterze: komunikat błędu i return.
}
```

**Number()** zamienia tekst z select na liczbę. Dla pustej opcji Number('') daje 0, więc warunek value < 1 wykrywa brak wyboru. **||** oznacza „lub”: wystarczy jedna prawdziwa część warunku. **return** kończy funkcję po błędzie.

## Zadanie 3. Połącz dane z widokiem

**1. Zachowaj wcześniejszą walidację.**
Dopisuj kod za blokiem if. Dzięki return żadna niepoprawna wartość nie trafi do listy.

**2. Usuń stary komunikat.**
Sprawdź, czy po poprawnym wyborze grade-message otrzymuje pusty textContent; to poprawka z rozgrzewki.

**3. Dodaj liczbę.**
Wywołaj push(value) na tablicy grades. Dodajesz zmienną value po konwersji Number, a nie tekst pobrany ponownie z select.

**4. Odśwież listę.**
Wywołaj renderGrades(). Samo push() zmienia dane w pamięci, ale nie aktualizuje strony.

## Sprawdź działanie

| Krok | Oczekiwany wynik |
|---|---|
| Dodaj 3, potem 4, potem 5 | Lista: 3, 4, 5; licznik: 3 |
| Ponownie dodaj 5 | Lista: 3, 4, 5, 5; licznik: 4 |
| Wybierz pustą opcję i dodaj | Błąd; lista i licznik bez zmian |
| Wybierz 2 i dodaj | Błąd znika; 2 pojawia się na końcu |

Przemyśl: dlaczego renderGrades() musi być po push(), a nie przed nim? Dlaczego nie potrzebujesz osobnej zmiennej przechowującej liczbę ocen?

# Usuń ostatnią ocenę

## pop() zmienia tablicę

```
const scores = [10, 20, 15];
const removed = scores.pop();
console.log(removed); // 15
console.log(scores); // [10, 20]
console.log(scores.length); // 2
```

**pop()** usuwa ostatni element i zwraca jego wartość. W naszej aplikacji nie musisz zapamiętywać zwracanej wartości. Wystarczy wywołanie metody, a później odświeżenie widoku.

```
const scores = [];
scores.pop();
console.log(scores.length); // nadal 0
```

Usunięcie z pustej tablicy nie powoduje błędu; pop() zwraca wtedy undefined. Tablica pozostaje pusta. Dzięki licznikowi odczytanemu z length nigdy nie zobaczysz ujemnej liczby ocen.

## Zadanie 4. Napisz removeLastGrade()

**1. Zmień dane.**
Wywołaj pop() na wspólnej tablicy grades. Przycisk ma usuwać wyłącznie ostatnią ocenę, niezależnie od bieżącego wyboru w select.

**2. Odśwież ekran.**
Wywołaj tę samą funkcję renderGrades(), której używasz przy dodawaniu. Nie kopiuj do removeLastGrade() kodu odpowiedzialnego za tekst listy i licznika.

**3. Przetestuj granicę.**
Odśwież stronę, dodaj 3 i 5. Usuń ostatnią ocenę: zostaje 3, licznik 1. Usuń ponownie: „Brak ocen”, licznik 0. Kliknij jeszcze raz: nadal ten sam stan.

## Jedno miejsce do wyświetlania

**Schemat działania**
addGrade() → push() → renderGrades()
removeLastGrade() → pop() → renderGrades()

Dwie akcje zmieniają dane, a jedna wspólna funkcja odpowiada za ich wyświetlanie. Za chwilę do tej funkcji podłączysz także statystyki.

Wyjaśnij własnymi słowami różnicę między zmianą tablicy przez push/pop a tworzeniem tekstu przez join. Które z tych działań modyfikują zawartość tablicy?

# Odwiedź każdą ocenę

W tabeli procentów pętla zwiększała procent o 10. Teraz pętla będzie zwiększać **indeks** o 1, aby odczytać każdy element tablicy.

```
const prices = [10, 20, 15];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total += prices[i];
}
console.log(total); // 45
```

| Fragment | Znaczenie |
|---|---|
| let i = 0 | Start od pierwszego indeksu |
| i < prices.length | Działaj, dopóki indeks mieści się w tablicy |
| i++ | Zwiększ indeks o 1 po każdym obiegu |
| prices[i] | Odczytaj element o bieżącym indeksie |
| total += prices[i] | Dodaj element do dotychczasowej sumy |

**let** pozwala przypisywać zmiennej nowe wartości. Zapis total += prices[i] oznacza to samo co total = total + prices[i]. Zmienna total jest akumulatorem: zbiera wynik kolejnych obiegów.

| i | Odczytana wartość | Suma przed | Suma po |
|---|---|---|---|
| 0 | 10 | 0 | 10 |
| 1 | 20 | 10 | 30 |
| 2 | 15 | 30 | 45 |

## Zadanie 5. Zacznij renderStats()

Na początku funkcji utwórz **let sum = 0**. Następnie napisz pętlę for przechodzącą po grades: start od i = 0, warunek i < grades.length, krok i++. W pętli dodawaj grades[i] do sum. Na próbę wypisz sum w konsoli.

Dla ocen 3, 4, 5 suma powinna wynosić 12. Na tym etapie możesz wywołać renderStats() w konsoli. Przy pustej tablicy pętla nie wykona się ani razu i suma wyniesie 0.

**Dwie ważne granice**
Użyj <, a nie <=: ostatni indeks jest o 1 mniejszy od length. Sumę ustawiaj na 0 wewnątrz renderStats(), przed pętlą: każde przeliczenie musi zacząć od nowa.

# Oblicz średnią bez dzielenia przez zero

Średnia arytmetyczna to suma ocen podzielona przez ich liczbę. Dla 3, 4, 5 otrzymujesz 12 / 3 = 4. Dla 3 i 4 jest to 7 / 2 = 3,5.

## Zadanie 6. Rozbuduj renderStats()

**1. Po pętli sprawdź długość.**
Jeżeli grades.length === 0, ustaw textContent elementu average na „—”. Ustaw też best-grade na „—” i zakończ funkcję przez return. Za chwilę dodasz jeszcze licznik ocen co najmniej 4.

**2. Dla niepustej tablicy oblicz średnią.**
Za warunkiem z return podziel sum przez grades.length. Pokaż wynik w elemencie average, używając istniejącej funkcji formatNumber().

**3. Sprawdź format.**
formatNumber() jest już gotowa. Pokazuje przecinek dziesiętny i maksymalnie dwa miejsca po przecinku, np. 3,5 lub 3,67. Do funkcji przekazuj liczbę po obliczeniu średniej.

```
// Przykład użycia gotowej funkcji:
const total = 11;
const count = 3;
console.log(formatNumber(total / count)); // 3,67
```

Nie dziel przez grades.length, zanim wykluczysz pustą tablicę. Przy pustej liście obliczenie 0 / 0 daje NaN, czyli wynik niebędący poprawną liczbą.

## Identyfikatory podsumowania

| id | Co ma pokazać |
|---|---|
| average | Średnią lub — przy braku ocen |
| best-grade | Najwyższą ocenę lub — przy braku ocen |
| good-count | Liczbę ocen >= 4; na pustej liście 0 |

## Sprawdź przed dalszym krokiem

Dla pustej listy wywołaj renderStats() w konsoli: średnia ma być „—”. Dodaj 3 i 4, ponownie wywołaj funkcję: średnia 3,5. Dopisz 4 i przelicz jeszcze raz: 3,67. Wielokrotne wywołanie bez zmiany listy ma dawać ten sam wynik.

Automatyczne wywołanie po dodawaniu i usuwaniu podłączysz w zadaniu 8. Najwyższa ocena i licznik zostaną obliczone w następnym zadaniu.

# Najwyższa ocena i licznik

## Szukanie największej wartości

Przechodząc przez tablicę, pamiętaj największą dotychczas znalezioną liczbę. Jeśli kolejna wartość jest większa, zapisz ją w tej zmiennej.

```
const prices = [10, 25, 15];
let highest = 0;
for (let i = 0; i < prices.length; i++) {
  if (prices[i] > highest) {
    highest = prices[i];
  }
}
console.log(highest); // 25
```

W projekcie oceny wynoszą 1-6, więc 0 może być wartością startową. Nie pokazuj jednak 0 jako najwyższej oceny pustej listy: w tym przypadku obowiązuje „—”.

## Zadanie 7. Dopisz dwa obliczenia do tej samej pętli

**1. Przygotuj zmienne.**
Obok sum, na początku renderStats(), utwórz let best = 0 i let good = 0. Za każdym wywołaniem mają zaczynać od zera.

**2. Zapamiętaj najwyższą ocenę.**
W istniejącej pętli, oprócz dodawania do sum, sprawdź, czy grades[i] jest większe od best. Jeśli tak, przypisz tę ocenę do best.

**3. Zlicz oceny co najmniej 4.**
Dodaj osobny if: jeśli grades[i] >= 4, zwiększ good o 1 przez good++. Nie używaj else if: ocena może jednocześnie być nowym maksimum i należeć do zliczanych ocen.

**4. Pokaż wyniki w odpowiedniej kolejności.**
Po pętli wpisz good do good-count, jeszcze przed warunkiem pustej tablicy. Dzięki temu pusta lista pokaże 0. Za warunkiem z return pokaż best w best-grade oraz średnią w average.

**Prześledź przykład**
Dla 6, 2, 4: suma 12, liczba ocen 3, średnia 4, najwyższa 6, oceny co najmniej 4: 2. Próg >= 4 dotyczy wyłącznie tego licznika; próg zaliczenia sprawdzianu pozostaje 40%.

# Aktualizuj wszystkie wyniki

## Zadanie 8. Podłącz renderStats()

Na końcu renderGrades(), **po** wyświetleniu listy i licznika, dodaj wywołanie renderStats(). Wywołanie ma wykonać się zarówno dla listy pustej, jak i niepustej. W renderGrades() nie dodawaj return, który ominąłby statystyki.

**Przepływ programu**
Kliknięcie → addGrade() lub removeLastGrade()
Zmiana grades → renderGrades() → renderStats()

Zmiana danych odbywa się raz. Następnie strona odczytuje ich aktualny stan. Nie musisz osobno wywoływać renderStats() z obu przycisków.

## Test całej ścieżki - bez odświeżania

Zacznij od pustej listy po odświeżeniu. Wykonuj kolejne kroki w podanej kolejności. Każda wartość w tabeli ma pojawić się automatycznie po kliknięciu.

| Akcja | Lista / liczba | Średnia | Max | >= 4 |
|---|---|---|---|---|
| Dodaj 3 | 3 / 1 | 3 | 3 | 0 |
| Dodaj 4 | 3, 4 / 2 | 3,5 | 4 | 1 |
| Dodaj 5 | 3, 4, 5 / 3 | 4 | 5 | 2 |
| Usuń ostatnią | 3, 4 / 2 | 3,5 | 4 | 1 |
| Usuń ostatnią | 3 / 1 | 3 | 3 | 0 |
| Usuń ostatnią | Brak ocen / 0 | — | — | 0 |
| Usuń ponownie | Brak ocen / 0 | — | — | 0 |
| Dodaj 6 | 6 / 1 | 6 | 6 | 1 |

## Dodatkowe próby

Przy niepustej liście wybierz pustą opcję i kliknij dodawanie: błąd ma się pojawić, a lista i wszystkie statystyki pozostać bez zmian. Następnie dodaj poprawną ocenę: błąd znika, podsumowanie się aktualizuje.

Sprawdź powtórzenia: 5 i 5 to dwie oceny, średnia 5, najwyższa 5, licznik ocen >= 4 równy 2. Dla 1, 2, 3 średnia wynosi 2, najwyższa 3, a licznik 0.

# Sprawdź i wyjaśnij swój kod

## Lista kryteriów ukończenia

| Obszar | Co ma działać |
|---|---|
| Przechowywanie | Jedna tablica grades poza funkcjami; zawiera liczby |
| Dodawanie | Walidacja, znikający błąd, push i odświeżenie widoku |
| Usuwanie | pop usuwa ostatni element; pusta lista jest bezpieczna |
| Widok | join i length pokazują listę oraz jej aktualny rozmiar |
| Statystyki | Pętla liczy sumę, maksimum i liczbę ocen >= 4 |
| Pusta lista | Brak ocen; liczba 0; średnia —; najwyższa —; >= 4: 0 |
| Powiązanie | renderGrades zawsze wywołuje renderStats |
| Wcześniejszy kod | Tabela i oba kalkulatory nadal działają; 40% zalicza |

## Gdy wynik jest nieprawidłowy

**Zostaje tylko ostatnia ocena:** sprawdź, czy tablica nie jest tworzona w addGrade(). **Oceny się sklejają zamiast sumować:** sprawdź Number() przy pobieraniu wartości. **Suma rośnie przy każdym odświeżeniu widoku:** sum, best i good powinny powstawać od nowa wewnątrz renderStats().

**NaN:** sprawdź warunek pustej tablicy i granicę i < grades.length. **Stare maksimum po usunięciu:** licz je ponownie z obecnej tablicy. **Statystyki nie znikają po usunięciu ostatniej oceny:** upewnij się, że renderStats() jest wywoływane również dla pustej listy.

## Wyjaśnij prowadzącemu

1. Dlaczego pierwszy indeks to 0, a długość pustej tablicy też wynosi 0?
2. Co zmieniają push i pop, a co zwraca join?
3. Dlaczego używasz let do sumy, a const do tablicy?
4. Dlaczego warunki maksimum i licznika to dwa osobne if?
5. W którym miejscu zabezpieczasz dzielenie przez zero?

**Oddaj działający projekt**
Przekaż index.html, styles.css i własny script.js. Pokaż test dodawania i usuwania aż do pustej listy oraz walidację pustego wyboru. Całą nową funkcjonalność wykonujesz w JavaScript, korzystając z gotowych elementów HTML.

Narzędzia lekcji: [] · indeks [i] · length · push() · pop() · join() · for · += · ++ · if · return.